

**A PROJECT REPORT ON CUSTOMER RELATIONSHIP MANAGEMENT
(CRM) SYSTEM *(Developed using Agile Scrum Methodology)***

Submitted By: Neeraj Sahu

Abstract

The modern business landscape requires agility and precision in managing customer interactions. Traditional "Waterfall" software development models often fail to deliver CRM systems that match the rapidly changing needs of sales and support teams. This project proposes the development of a **Customer Relationship Management (CRM)** system using the **Agile Scrum Methodology**.

Unlike conventional systems that are rigid and monolithic, this CRM is built iteratively. The project is divided into 2-week "Sprints," ensuring that critical features like Lead Management and Support Ticketing are delivered and tested early. The system aims to solve the problem of fragmented customer data by providing a centralized "Single Source of Truth" for sales representatives.

Technical implementation utilizes a specific tech stack (Java/Kotlin for backend logic, SQL for data storage) to ensure robustness. The final deliverable is not just a software application but a documented case study on how Agile ceremonies (Sprint Planning, Daily Stand-ups, Retrospectives) significantly reduce development risk and improve product quality.

Keywords: Agile, Scrum, CRM, Sprint, Lead Management, Software Engineering.

Introduction

1.1 Introduction

In an era where data is the new oil, managing customer relationships manually via spreadsheets is a liability. A CRM system is the backbone of any sales-driven organization. It captures leads, tracks follow-ups, and manages support tickets. This project focuses on building a "Lite" version of a CRM tailored for Small-to-Medium Enterprises (SMEs).

1.2 The "Why Agile?" Factor

Most academic projects follow the Waterfall model: Requirements -> Design -> Code -> Test. This fails in the real world because requirements change.

We adopted Agile Scrum for this project to:

- **Mitigate Risk:** By building the "Login" and "Database" modules first (Sprint 1), we validated our architecture early.
- **Feedback Loop:** Each feature was reviewed at the end of every sprint, allowing for immediate corrections.

1.3 Problem Identification (The "Pain Points")

Current manual systems suffer from:

1. **Data Silos:** Sales team has one Excel sheet; Support team has another. No data syncing.
2. **Lead Leakage:** Without automated reminders, sales reps forget to follow up on potential clients.
3. **Lack of Visibility:** Managers cannot see which sales rep is performing well without manually calculating reports.

1.4 Project Objectives

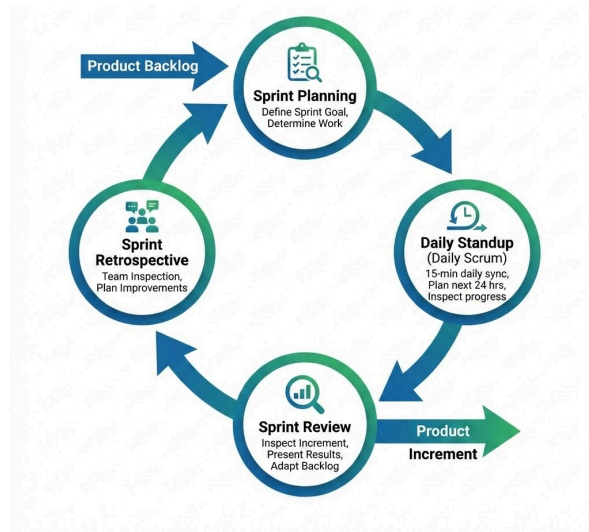
- To develop a web-based CRM accessible to field agents.
- To implement **Role-Based Access Control (RBAC)** ensuring data security (Admins see all; Agents see only their leads).
- To utilize **Jira/Trello** for tracking project velocity and burn-down rates.

Agile Methodology & Process

2.1 The Scrum Framework

We utilized the standard Scrum lifecycle. The project timeline was divided into three distinct Sprints.

- **Product Backlog:** A master list of all desired features (e.g., "Export to PDF", "Dark Mode").
- **Sprint Backlog:** A smaller list of tasks selected for *this* specific 2-week cycle.



2.2 Sprint Breakdown (The Execution Plan)

Sprint	Duration	Focus Area	Key Deliverables
Sprint 1	Week 1-2	Architecture	Database Schema, User Login (Auth), Dashboard Layout
Sprint 2	Week 3-4	Sales Module	Add Leads, Edit Customer, "My Pipeline" View
Sprint 3	Week 5-6	Support & Reporting	Ticketing System, Admin Charts, Final Bug Fixes

2.3 Roles in this Project

- **Product Owner:** Defined *what* to build (e.g., "We need a 'Lost Reason' dropdown for failed sales").
- **Scrum Master:** Ensured the team stuck to the timeline and removed technical blockers.
- **Dev Team:** Handled the Full-Stack implementation (Frontend + Backend + Database).

Software Requirement Specification (SRS)

3.1 Functional Requirements

These are the core behaviors of the system:

1. **FR-01 (Auth):** System must validate user credentials against encrypted passwords in the DB.
2. **FR-02 (Lead Gen):** Sales reps must be able to Create, Read, Update, and Delete (CRUD) lead records.
3. **FR-03 (Search):** Users must be able to search for customers by Phone Number or Email ID.
4. **FR-04 (Security):** A "Sales Rep" should not be able to delete an "Admin" account.

3.2 Non-Functional Requirements

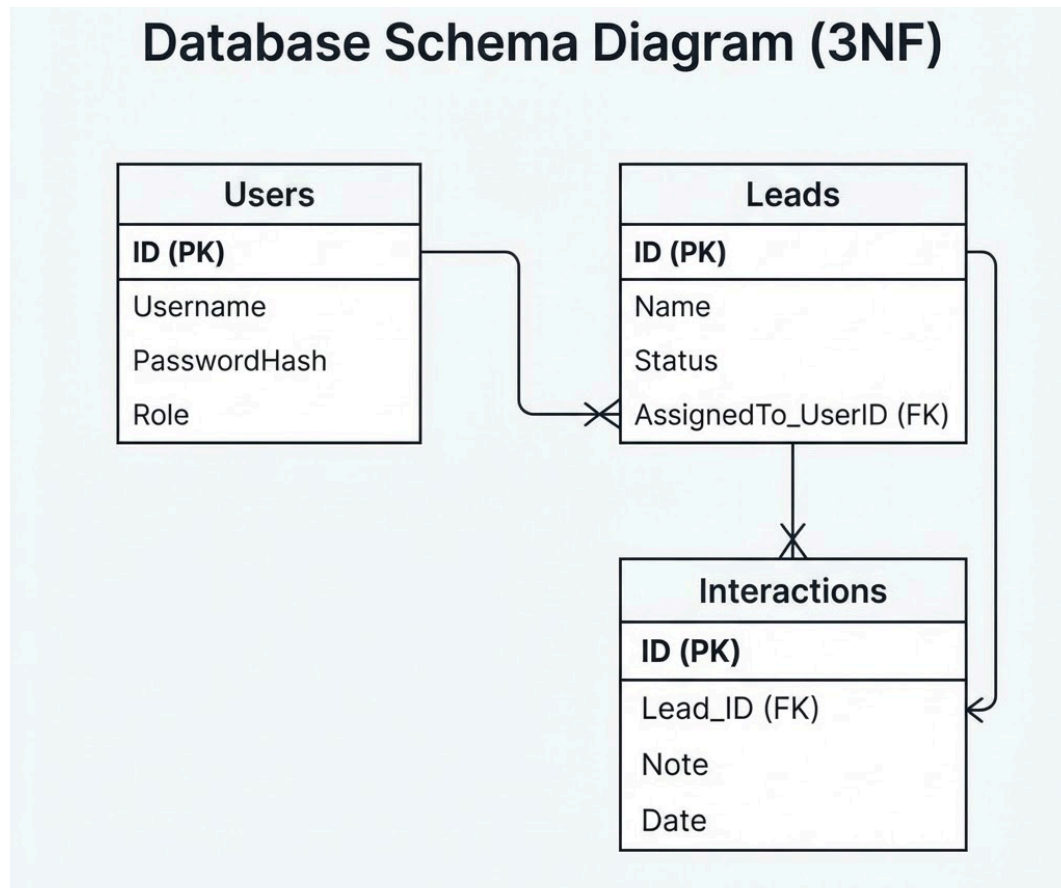
1. **Performance:** The dashboard must load within 2 seconds under a load of 50 concurrent users.
2. **Scalability:** The database design must support up to 10,000 customer records without degradation.
3. **Availability:** The system should be available 99.9% of the time during business hours.

3.3 Technology Stack

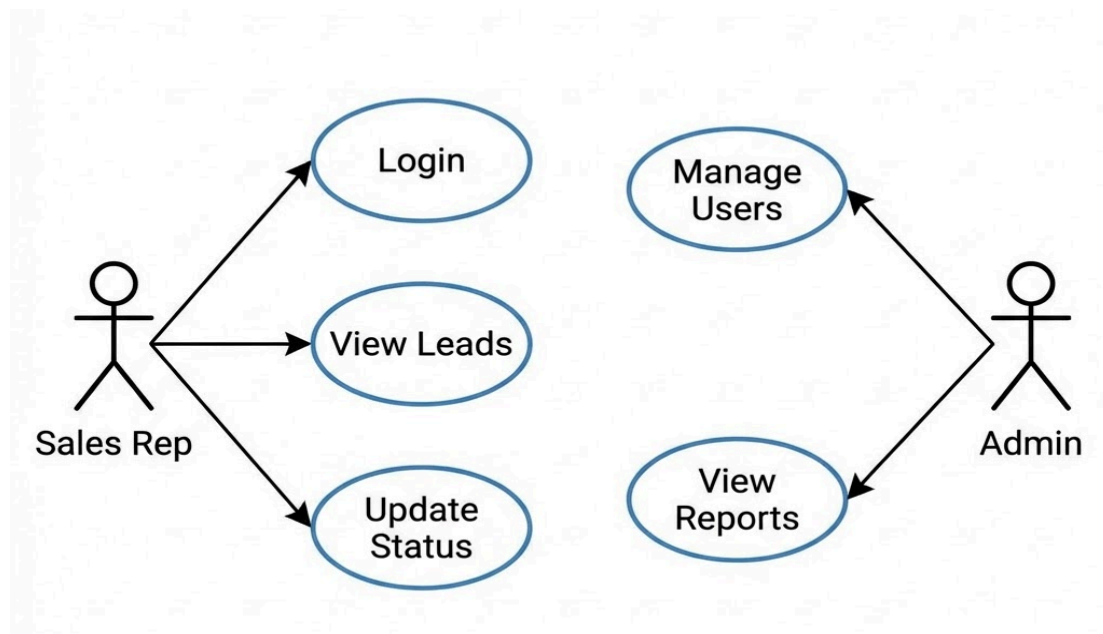
- **Frontend:** HTML5, CSS3, JavaScript (React/Angular for dynamic tables).
- **Backend:** Java (Spring Boot) or Node.js.
- **Database:** MySQL (Relational Data Model).
- **Testing Tool:** Selenium Webdriver (for automation) & JUnit.

System Design

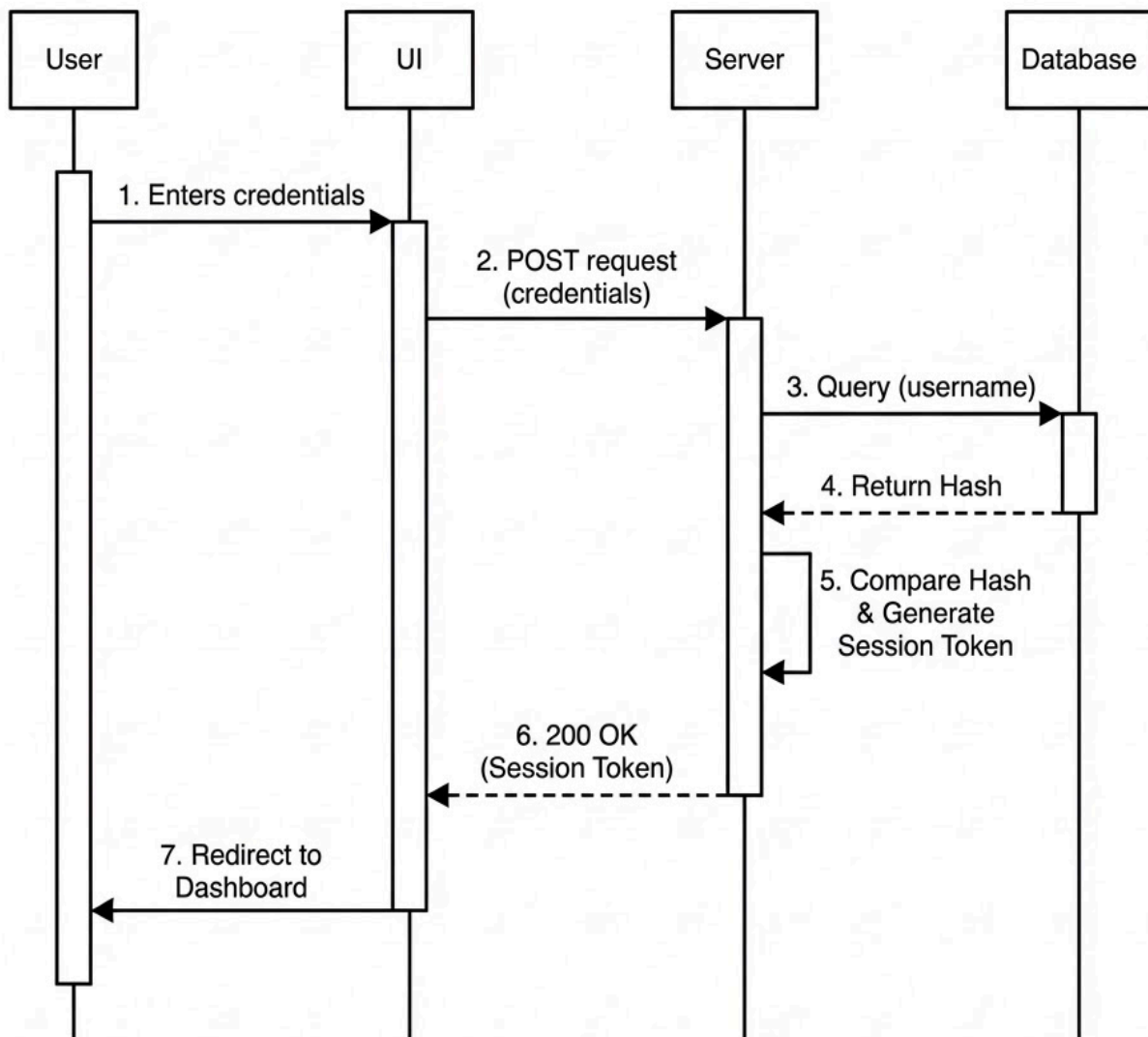
4.1 Database Schema (ER Model)



4.2 Use Case Diagram



4.3 Sequence Diagram (Login Flow)



Sprint Execution Report

5.1 Sprint 1 Retrospective

- **Planned:** Setup Login and Database.
- **Achieved:** 100% completion.
- **Challenges:** JDBC connection issues initially caused delays. Fixed by reconfiguring the connection pool.

5.2 Sprint 2 Retrospective (The "Sales" Module)

- **Planned:** "Add Customer" and "Edit Customer" screens.
- **Deviation:** We realized we forgot a "Search" bar. We added it to the Sprint Backlog mid-sprint as a "Change Request."
- **Agile Benefit:** Because we were Agile, we could add the Search bar immediately without waiting for a 6-month cycle to end.

5.3 Sprint 3 Retrospective (Reporting)

- **Planned:** Admin Dashboard.
- **Outcome:** Created a dynamic pie chart showing "Leads Won vs. Lost".
- **Status:** Project marked as "Release Ready".

Testing & Quality Assurance

6.1 Testing Strategy

We adopted a "Shift Left" testing strategy, meaning testing happened inside the sprint, not just at the end.

6.2 Test Cases (Sample)

TC ID	Scenario	Pre-Condition	Steps	Expected Result	Status
TC_01	Login Valid	User Registered	Enter valid User/Pass	Redirect to Dash	PASS
TC_02	SQL Injection	Login Page	Enter ' OR 1=1 --	System Block Access	PASS
TC_03	Role Access	Logged as Rep	Try accessing /admin	"Access Denied" Error	PASS
TC_04	Lead Status	Lead exists	Change status to "Closed"	DB updates status	PASS

Future Scope & Conclusion

7.1 Limitations

- **Offline Mode:** Currently, the system requires active internet. It does not work offline.
- **Mobile Support:** The UI is responsive but not a native Android/iOS app.

7.2 Future Enhancements

1. **AI Integration:** Using Machine Learning to predict which leads are most likely to buy based on past data.
2. **WhatsApp API:** Integrating WhatsApp Business API to send automated follow-ups directly from the CRM.
3. **Voice Notes:** Allowing sales reps to record audio notes after a meeting instead of typing.

7.3 Conclusion

This project successfully demonstrated that Agile Methodology is superior for developing business applications like CRM. By breaking the project into sprints, we avoided the "Analysis Paralysis" of Waterfall. The final product is a functional, tested, and user-centric application that solves the core problem of lead mismanagement.

References

1. Beck, K., et al. (2001). *The Agile Manifesto*. Agile Alliance.
2. Schwaber, K. (2004). *Agile Project Management with Scrum*. Microsoft Press.
3. Gamma, E., et al. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*.
4. Standard SQL Documentation (MySQL Reference Manual).
5. *Internal Lecture Notes on Software Engineering*, Technocrats Institute of Technology, Bhopal.